

目录

定位设备数据对接技术文档	1
1. 服务请求和响应	1
1.1 描述:	1
1.2 证书管理:	1
1.3 Base URLs:	1
2. 流程示意图	2
3. 客户接口	2
2.1 发送请求绑定设备	2
● 请求格式描述:	2
● 功能描述:	3
● 参数说明:	3
● PostMan 操作图, 如下:	4
● Example Request	5
● Example Success Response	5
● Example Failure Response	5
2.2 获取蓝牙设备绑定状态	7
● 请求格式描述:	7
● 功能描述:	7
● 参数说明	7
● PostMan 操作图, 如下:	8
● Example Request	9
● Example Success Response	9
● Example Failure Response	9
2.3 获取已绑定的蓝牙设备定位数据	11
● 请求格式描述:	11
● 功能描述:	12
● 参数说明:	12
● PostMan 操作图, 如下:	13
● Example Request	14
● Example Success Response	14
● Example Failure Response	16
2.4 检索已绑定的设备列表	17
● 请求格式描述:	17
● 功能描述:	18
● 参数说明:	18
● PostMan 操作图, 如下:	19
● Example Request	20
● Example Success Response	20
● Example Failure Response	21
2.5 设备请求解除绑定	22
● 请求格式描述:	22
● 功能描述:	23

● 参数说明:	23
● PostMan 操作图, 如下:	23
● Example Request.....	24
● Example Success Response.....	24
● Example Failure Response.....	25
4. 代码对接事例	26
3.1 Node. JS 语言代码对接.....	26
3.2 JAVA 语言代码对接.....	28
3.3 Python 语言代码对接.....	33
3.4 C# 语言代码对接.....	35
Status Codes	39

定位设备数据对接技术文档

注：版本记录

版本	变更内容	更新时间	描述
V1.1	提供证书访问接口	2025-03-26(中国北京时间)	
V1.2	增加代码对接事例	2025-03-26(中国北京时间)	
V1.3	忽略证书认证	2025-04-02(中国北京时间)	
V1.4	完善提示语	2025-04-08(中国北京时间)	
V1.5	完善参数说明	2025-06-06(中国北京时间)	
V1.6	完善参数说明	2025-06-08(中国北京时间)	

1. 服务请求和响应

1.1 描述：

- Licensee Server 与 服务器之间采用 SSL（Secure Sockets Layer）/ TLS（Transport Layer Security）协议 进行安全通信，确保数据的 机密性、完整性和身份验证。

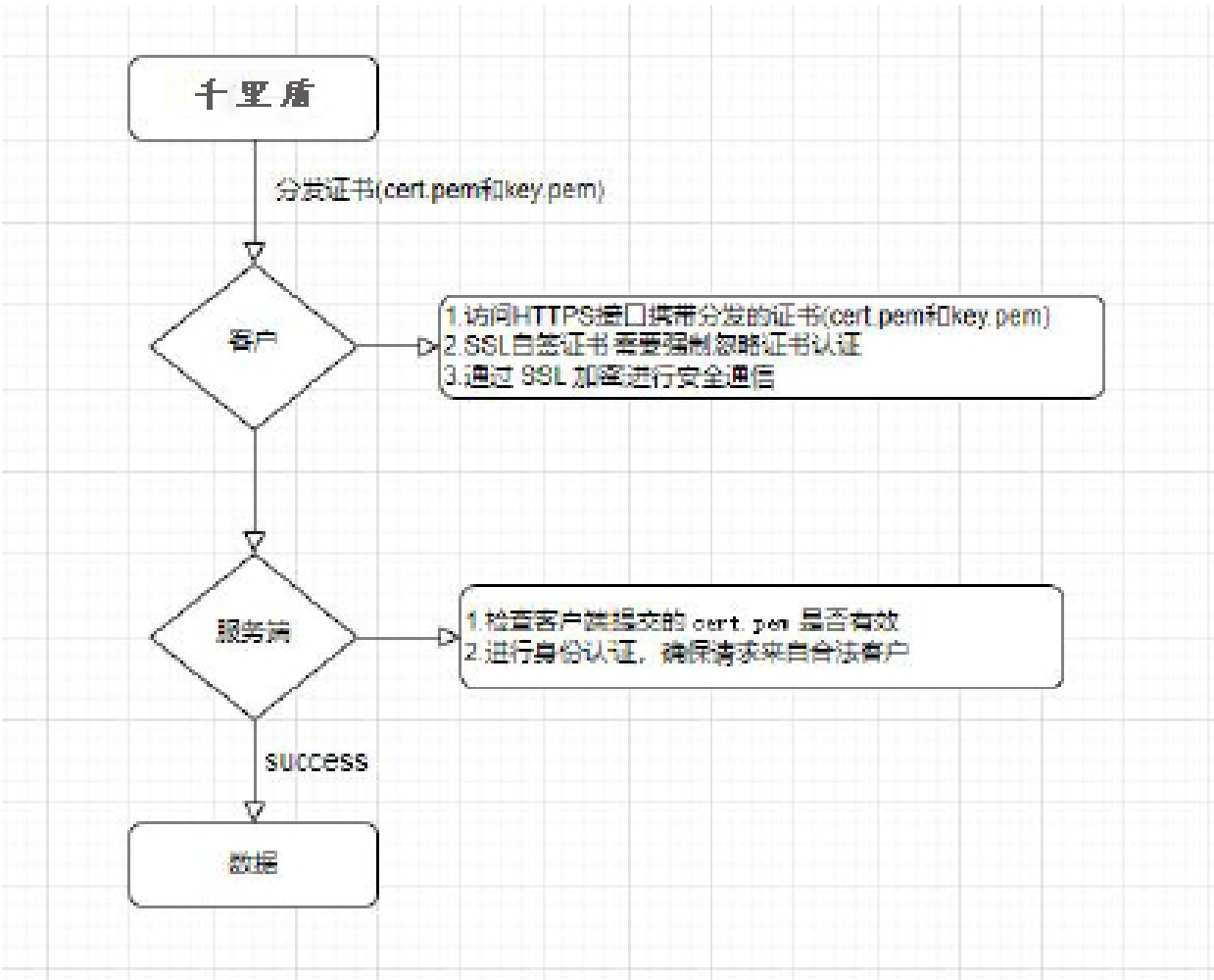
1.2 证书管理：

- 证书提供方：QLD提供 cert.pem 和 key.pem 证书。
- 证书用途：
客户端（Licensee Server）：
访问 HTTPS API 时，携带 cert.pem 和 key.pem 进行身份认证。

1.3 Base URLs：

- https://39.107.83.255:8444

2. 流程示意图



3. 客户接口

2.1 发送请求绑定设备

● 请求格式描述:

HTTP Method	POST
URI	/api/device/receiveDeviceDataBind
Header	Accept: application/json Content-Type: application/json User-Agent: Node.js
httpsAgent	提供 cert.pem 证书、key.pem 私钥
URL Params	None

Request (multipart/form-data)	<pre>{ "deviceName": "xxxxxx", "deviceId": "xxxxxx", "macAddress": "xxxxxx", "localName": "xxxxxx" }</pre>
Response	<pre>{ "code": xxx, "data": [], "message": "xxx" }</pre>

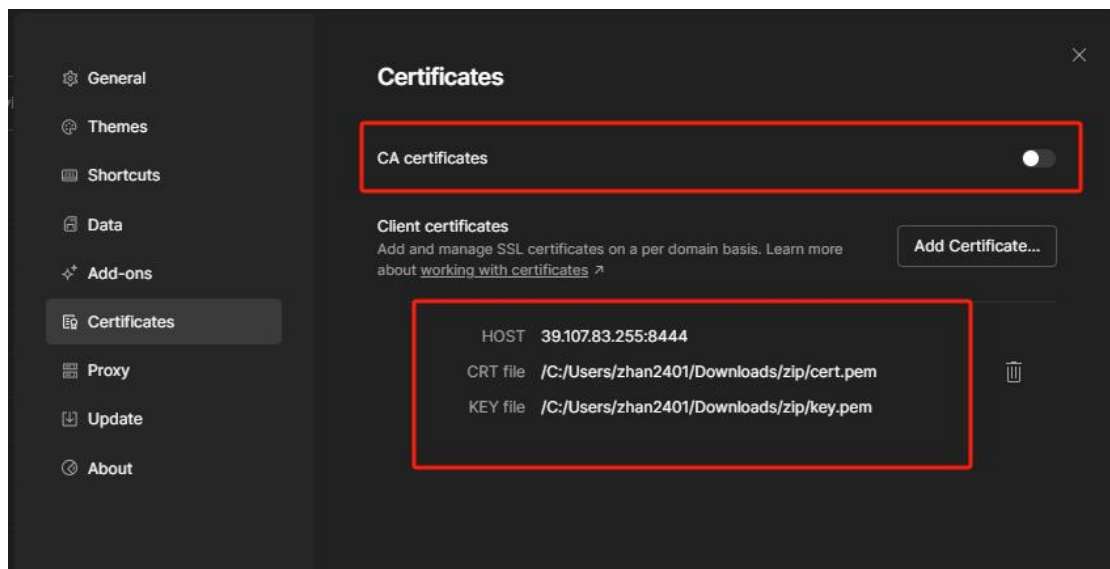
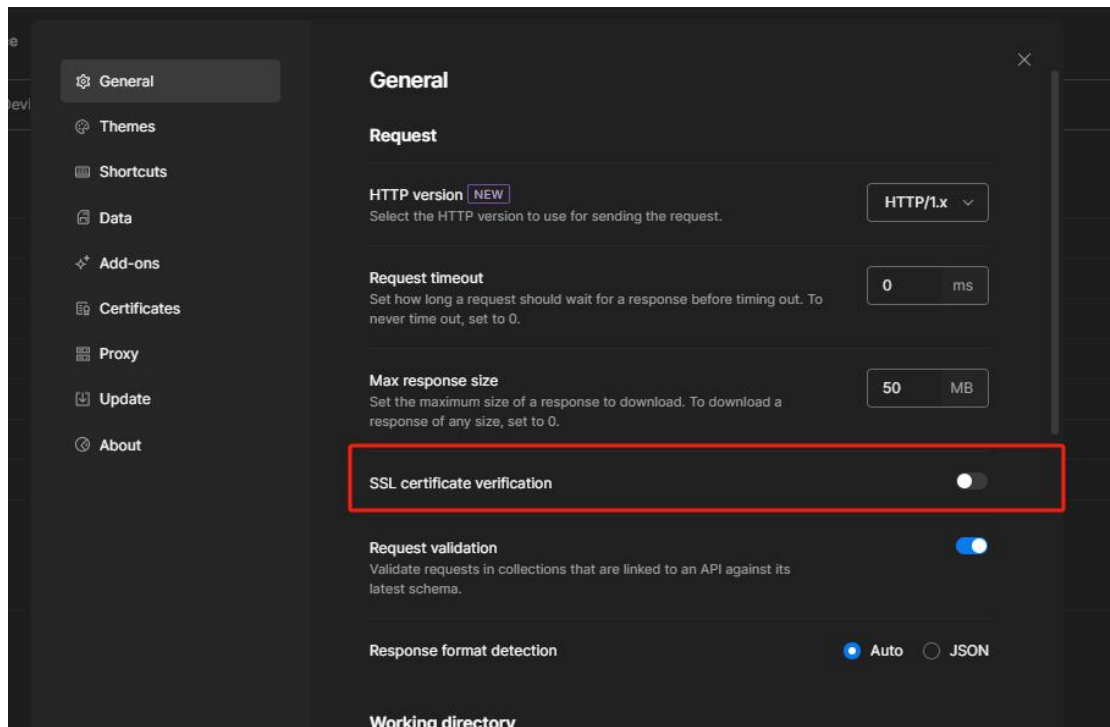
● 功能描述:

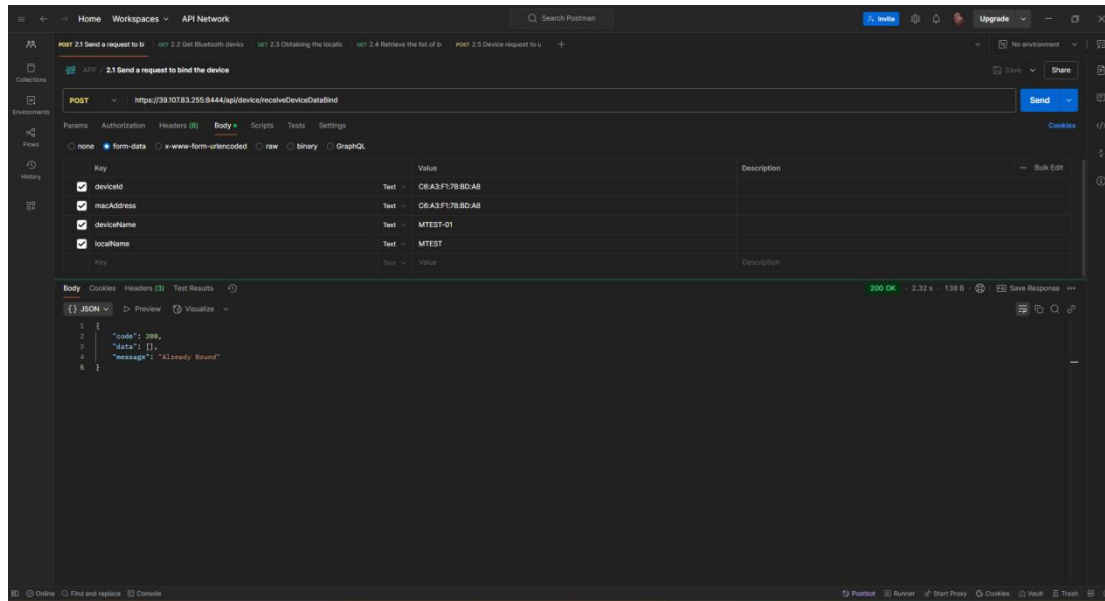
- A. 请求头携带 cert.pem 证书和 key.pem;
- B. 支持单设备绑定在该证书下;

● 参数说明:

参数名	参数说明	参数类型	是否必需
deviceName	设备名	字符串	是
deviceId	设备 id	字符串	是
macAddress	设备 mac 地址	字符串	是
localName	统一设备烧录 localName	字符串	是

- PostMan 操作图，如下：





● Example Request

<https://39.107.83.255:8444/api/device/receiveDeviceDataBind>

● Example Success Response

```
{
  "code": 200,
  "data": [],
  "message": "Already Bound"
}
```

● Example Failure Response

```
{
  "code": 500,
  "data": [],
  "message": "Device not found"
}
{
  "code": 500,
  "data": [],
  "message": "Device is already bound"
}
{
```

```

    "code": 500,
    "data": [],
    "message": "Not Found (bad URL)"
  }
  {
    "code": 401,
    "data": [],
    "message": "Unauthorized (this area requires you to log in)"
  }
  {
    "code": 405,
    "data": [],
    "message": "Method Not Allowed (wrong HTTP method)"
  }
  {
    "code": 404,
    "data": [],
    "message": "Not Found (bad URL)"
  }
  {
    "code": 400,
    "data": [],
    "message": "Bad Request (generic user error/bad data/invalid data)"
  }
  {
    "code": 500,
    "data": [],
    "message": "Bad Request (generic localName error/bad data/invalid data)"
  }
  {
    "code": 500,
    "data": [],
    "message": "Request Param Cannot Be Empty"
  }
  {
    "code": 500,
    "data": [],
    "message": "MAC address is invalid, and the account is not managed"
  }
}

```

2.2 获取蓝牙设备绑定状态

● 请求格式描述:

HTTP Method	GET
URI	/api/device/findByDeviceStatus?localName=xxx&macAddress=xxx
Header	Accept: application/json Content-Type: application/json
httpsAgent	提供 cert.pem 证书、key.pem 私钥
URL Params	"macAddress": "xxxx", "localName": "xxxx"
Request(multipart/form-data)	None
Response	{ "code": xxx, "data": xxx, "message": "xxx" }

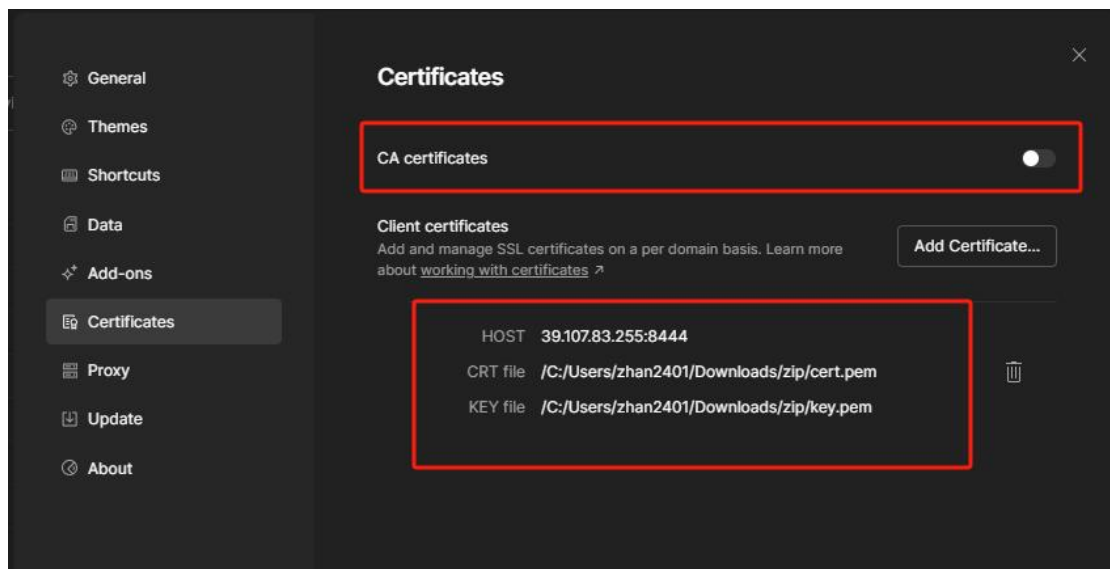
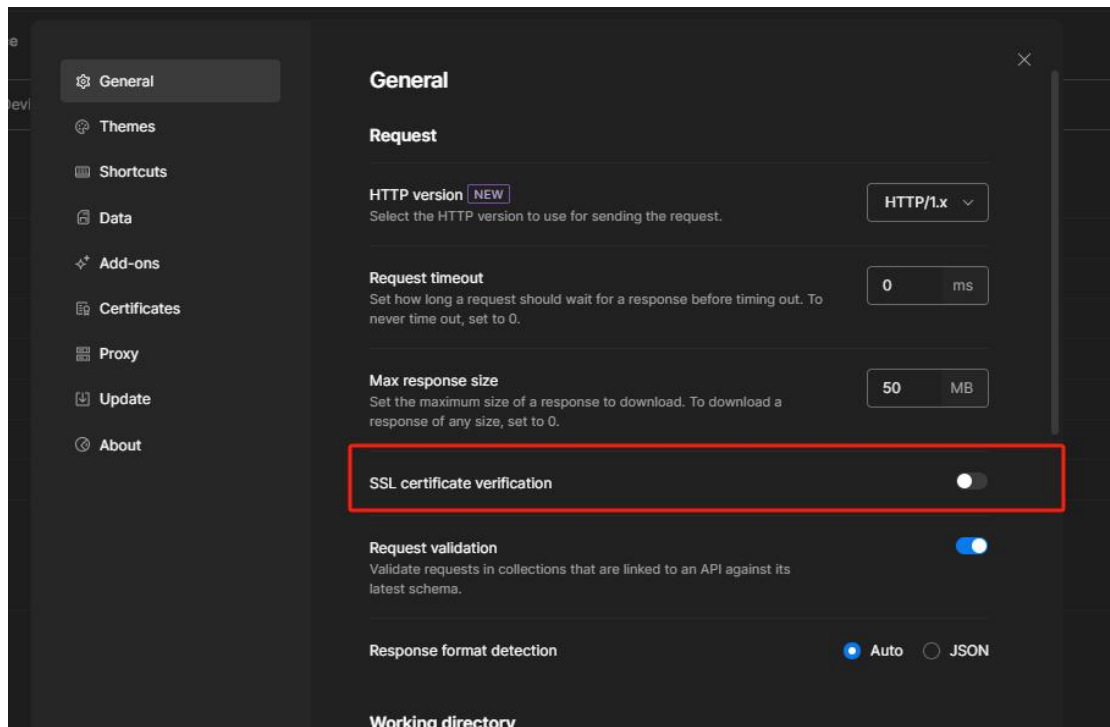
● 功能描述:

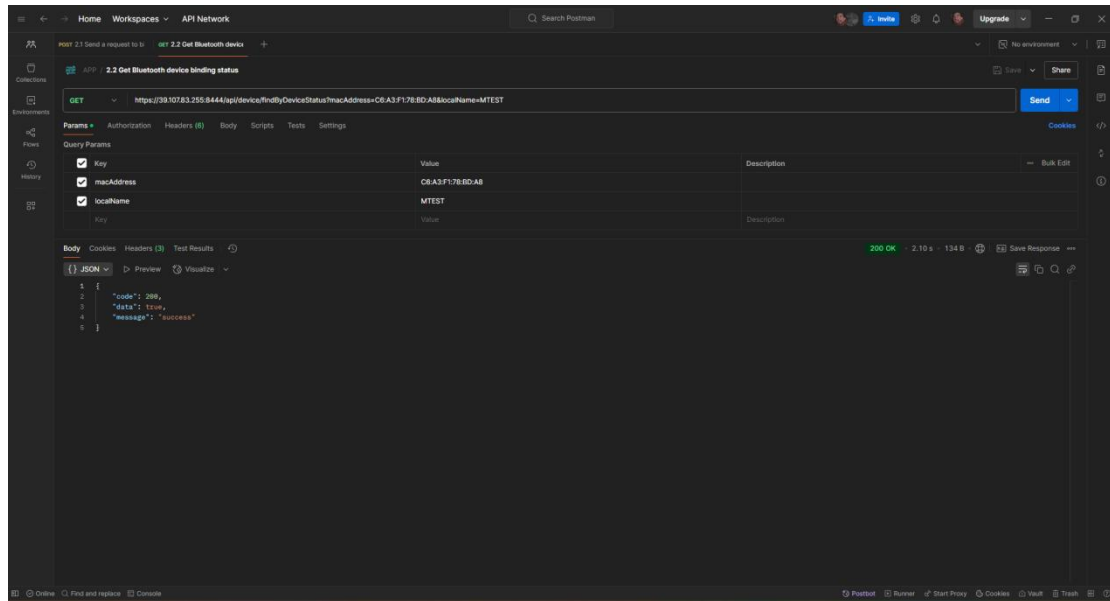
- A. 请求头携带 cert.pem 证书和 key.pem;
- B. 根据 mac 地址获取单设备的绑定状态, Data 返回参数为 true 则绑定成功;

● 参数说明

参数名	参数说明	参数类型	是否必需
macAddress	设备 mac 地址	字符串	是
localName	统一设备烧录 localName	字符串	是

- PostMan 操作图，如下：





● Example Request

`https://39.107.83.255:8444/api/device/findByDeviceStatus?macAddress=C6:A3:F1:78:BD:A8&localName=MTEST`

● Example Success Response

```
{
  "code": 200,
  "data": true,
  "message": "success"
}
```

● Example Failure Response

```
{
  "code": 500,
  "data": [],
  "message": "Request Param Cannot Be Empty"
}
{
  "code": 500,
  "data": [],
  "message": "not Device Data"
}
```

```

{
  "code": 500,
  "data": [],
  "message": "Not Found (bad URL)"
}
{
  "code": 401,
  "data": [],
  "message": "Unauthorized (this area requires you to log in)"
}
{
  "code": 405,
  "data": [],
  "message": "Method Not Allowed (wrong HTTP method)"
}
{
  "code": 404,
  "data": [],
  "message": "Not Found (bad URL)"
}
{
  "code": 400,
  "data": [],
  "message": "Bad Request (generic user error/bad data/invalid data)"
}
{
  "code": 500,
  "data": [],
  "message": "Bad Request (generic localName error/bad data/invalid data)"
}
{
  "code": 500,
  "data": [],
  "message": "Request Param Cannot Be Empty"
}
{
  "code": 500,
  "data": [],
  "message": "MAC address is invalid, and the account is not managed"
}

```

2.3 获取已绑定的蓝牙设备定位数据

● 请求格式描述:

HTTP Method	GET
URI	/api/device/findByBoundDeviceLocationData?localName=xxx&time=&deviceId=&page=1&size=10
Header	Accept: application/json Content-Type: application/json
httpsAgent	提供 cert.pem 证书、key.pem 私钥
URL Params	"localName": "xxxxxx", "time": "xxxxxx", "deviceId": "xxxxxx", "page": "xxxxxx", "size": "xxxxxx"
Request (multipart/form-data)	None
Response	<pre>{ "code": 200, "data": [{ "deviceId": "xxx", "deviceName": "xxx", "latitude": "xxx", "longitude": "xxx", "datePublished": "xxx", "timestamp": "xxx" }], "message": "success" }</pre> <pre>{ "code": 200, "data": [{ "deviceId": "xxx", "deviceName": "xxx", "latitude": "xxx", "longitude": "xxx", "datePublished": "xxx", "timestamp": "xxx" }] }</pre>

	<pre>], "pageData": { "totalPages": x, "size": x, "number": x, "numberOfElements": x, "totalElements": x }, "message": "success" } </pre>
--	--

● 功能描述:

- A. 请求头携带 cert.pem 证书和 key.pem;
- B. 以 (UTC)timestamp 设备采集数据时间为准
- C. 少量数据支持不传分页获取定位设备数据, 当返回定位设备数据量超过 500 条, 强制分页返回(每页条数最高 500 条, 自行叠加页数查询);
- D. 支持自定义分页数, 每页条数最高支持 500 条;
- E. 分页是为了避免数据过长导致客户端接收不完整, 以及导致查询过慢;

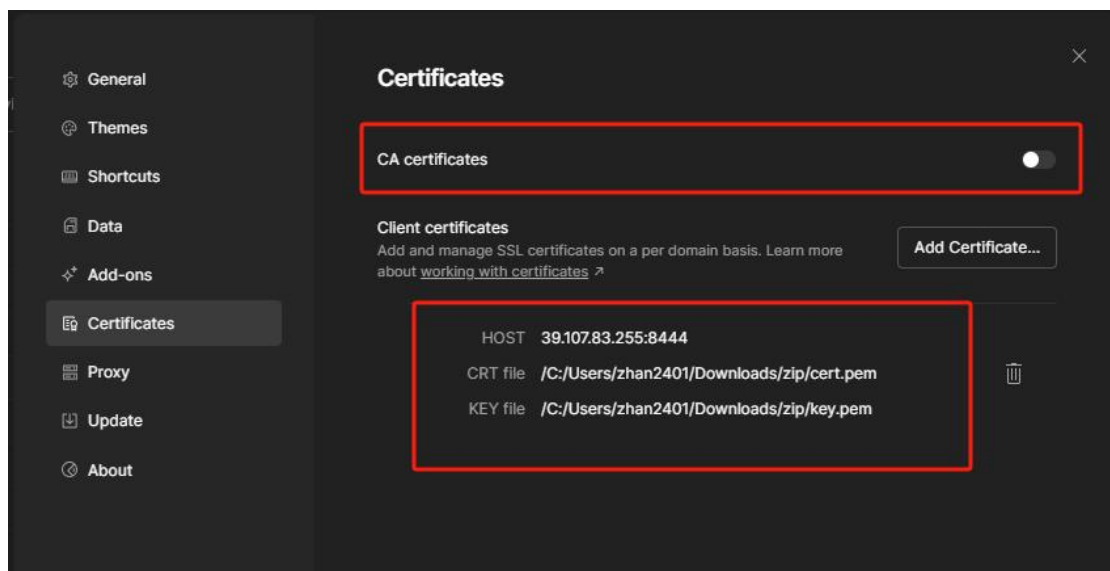
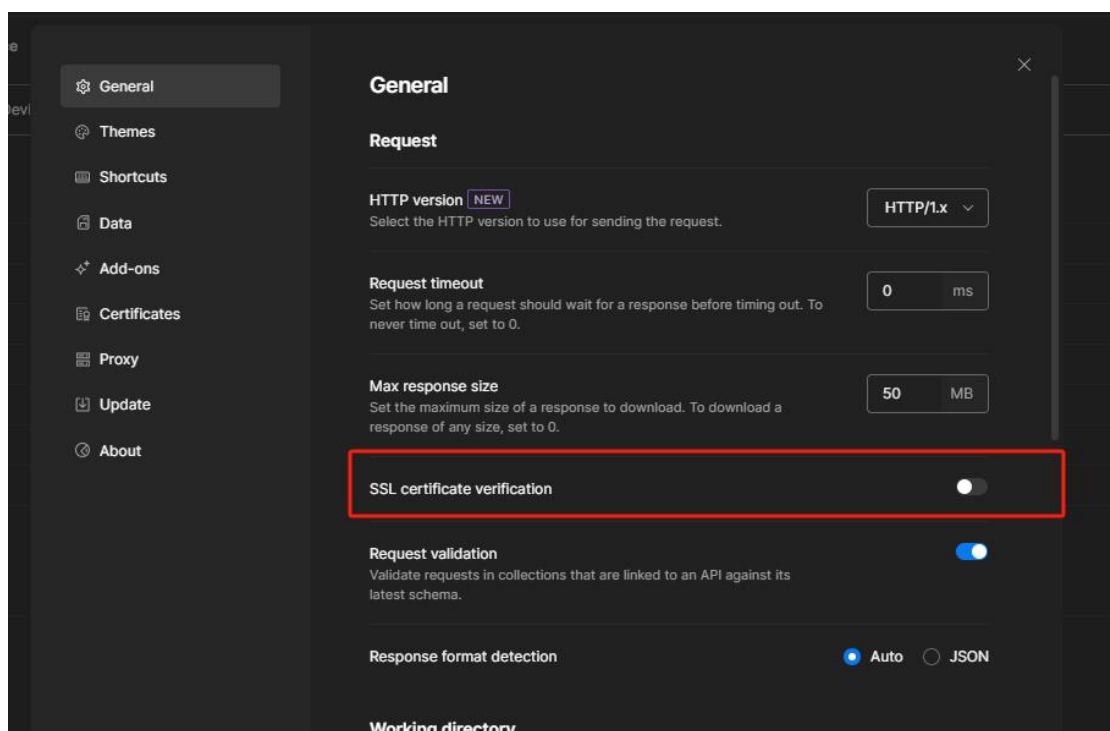
● 参数说明:

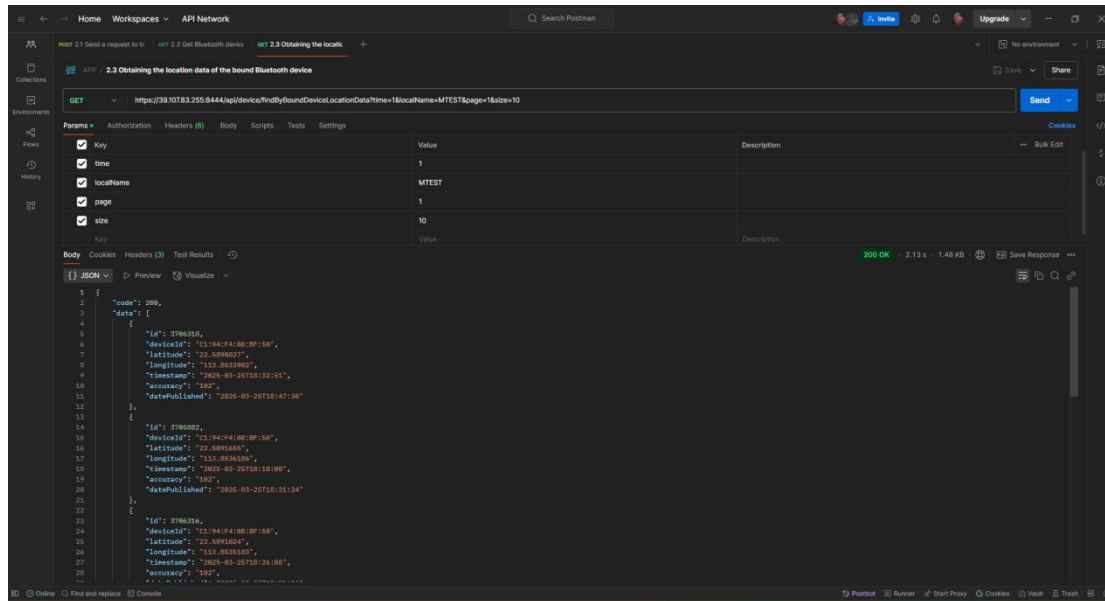
参数名	参数说明	参数类型	是否必需
localName	统一设备烧录 localName	字符串	是
time	时间 (小时为单位)	字符串	否
deviceId	设备 id	字符串	否
page	当前页数(默认第一页)	整数	是
size	当前页条数(默认每页 10 条)	整数	是

响应参数名	响应参数说明	参数类型
deviceId	设备 ID	字符串
deviceName	设备名称	字符串
latitude	纬度	字符串
longitude	经度	字符串
datePublished	UTC 数据上传时间 (Format: yyyy-MM-dd HH:mm:ss)	字符串
timestamp	UTC 时间戳 (毫秒) / UTC 设备采集数据时间 (Format: yyyy-MM-dd HH:mm:ss)	字符串
totalPages	总页数	整数
size	请求页_条数	整数
number	请求页数	整数

numberOfElements	当前页返回条数	整数
totalElements	总条数	整数

● PostMan 操作图，如下：





● Example Request

`https://39.107.83.255:8444/api/device/findByBoundDeviceLocationData?time=1&localName=MTEST&page=1&size=10`

● Example Success Response

```
{
  "code": 200,
  "data": [
    {
      "id": 3706318,
      "deviceId": "C1:94:F4:0B:BF:50",
      "latitude": "22.5890837",
      "longitude": "113.8533902",
      "timestamp": "2025-03-25T18:32:51",
      "accuracy": "102",
      "datePublished": "2025-03-25T18:47:30"
    },
    {
      "id": 3705882,
      "deviceId": "C1:94:F4:0B:BF:50",
      "latitude": "22.5891655",
      "longitude": "113.8536186",
      "timestamp": "2025-03-25T18:18:00",
      "accuracy": "102",
      "datePublished": "2025-03-25T18:31:24"
    }
  ]
}
```

```

},
{
  "id": 3706316,
  "deviceId": "C1:94:F4:0B:BF:50",
  "latitude": "22.5891024",
  "longitude": "113.8535183",
  "timestamp": "2025-03-25T18:26:08",
  "accuracy": "102",
  "datePublished": "2025-03-25T18:31:24"
},
{
  "id": 3706319,
  "deviceId": "C1:94:F4:0B:BF:50",
  "latitude": "22.5891655",
  "longitude": "113.8536186",
  "timestamp": "2025-03-25T18:20:59",
  "accuracy": "102",
  "datePublished": "2025-03-25T18:31:24"
},
{
  "id": 3705479,
  "deviceId": "C1:94:F4:0B:BF:50",
  "latitude": "22.5891166",
  "longitude": "113.8534743",
  "timestamp": "2025-03-25T18:04:45",
  "accuracy": "102",
  "datePublished": "2025-03-25T18:15:15"
},
{
  "id": 3705480,
  "deviceId": "C1:94:F4:0B:BF:50",
  "latitude": "22.5891188",
  "longitude": "113.8534097",
  "timestamp": "2025-03-25T18:09:51",
  "accuracy": "102",
  "datePublished": "2025-03-25T18:15:15"
},
{
  "id": 3706317,
  "deviceId": "C1:94:F4:0B:BF:50",
  "latitude": "22.5891166",
  "longitude": "113.8534743",
  "timestamp": "2025-03-25T18:03:10",
  "accuracy": "102",

```

```

        "datePublished": "2025-03-25T18:15:15"
      }
    ],
    "pageData": {
      "totalPages": 1,
      "size": 10,
      "number": 1,
      "numberOfElements": 7,
      "totalElements": 7
    },
    "message": "success"
  }
}

```

● Example Failure Response

```

{
  "code": 500,
  "data": [],
  "message": "parameter error"
}

{
  "code": 500,
  "data": [],
  "message": "Request Param Cannot Be Empty"
}

{
  "code": 500,
  "data": [],
  "message": "Not Found (bad URL)"
}

{
  "code": 401,
  "data": [],
  "message": "Unauthorized (this area requires you to log in)"
}

{
  "code": 405,
  "data": [],
  "message": "Method Not Allowed (wrong HTTP method)"
}

{
  "code": 404,

```

```

    "data": [],
    "message": "Not Found (bad URL)"
  }
  {
    "code": 400,
    "data": [],
    "message": "Bad Request (generic user error/bad data/invalid data)"
  }
  {
    "code": 500,
    "data": [],
    "message": "Bad Request (generic localName error/bad data/invalid data)"
  }
  {
    "code": 500,
    "data": [],
    "message": "Request Param Cannot Be Empty"
  }
  {
    "code": 500,
    "data": [],
    "message": "MAC address is invalid, and the account is not managed"
  }
}

```

2.4 检索已绑定的设备列表

● 请求格式描述:

HTTP Method	GET
URI	/api/device/findByDeviceStatusAll?localName=xxx&page=1&size=10
Header	Accept: application/json Content-Type: application/json
httpsAgent	提供 cert.pem 证书、key.pem 私钥
URL Params	"localName": "xxxxxx", "page": "xxxxxx", "size": "xxxxxx"
Request (multipart/form-data)	None
Response	{

	<pre> "code": 200, "data": [{ "macAddress": "xxx", "deviceName": "xxx", "deviceId": "xxx", "status": xxx }], "pageData": { "totalPages": xxx, "size": xxx, "number": xxx, "numberOfElements": xxx, "totalElements": xxx }, "message": "success" } </pre>
--	--

● 功能描述:

- A. 请求头 cert.pem 证书和 key.pem;
- B. 少量数据支持不传分页获取设备列表, 当返回设备量超过 500 条, 强制分页返回(每页条数最高 500 条, 自行叠加页数查询);
- C. 支持自定义分页数, 每页条数最高支持 500 条;
- D. 分页是为了避免数据过长导致客户端接收不完整, 以及导致查询过慢;

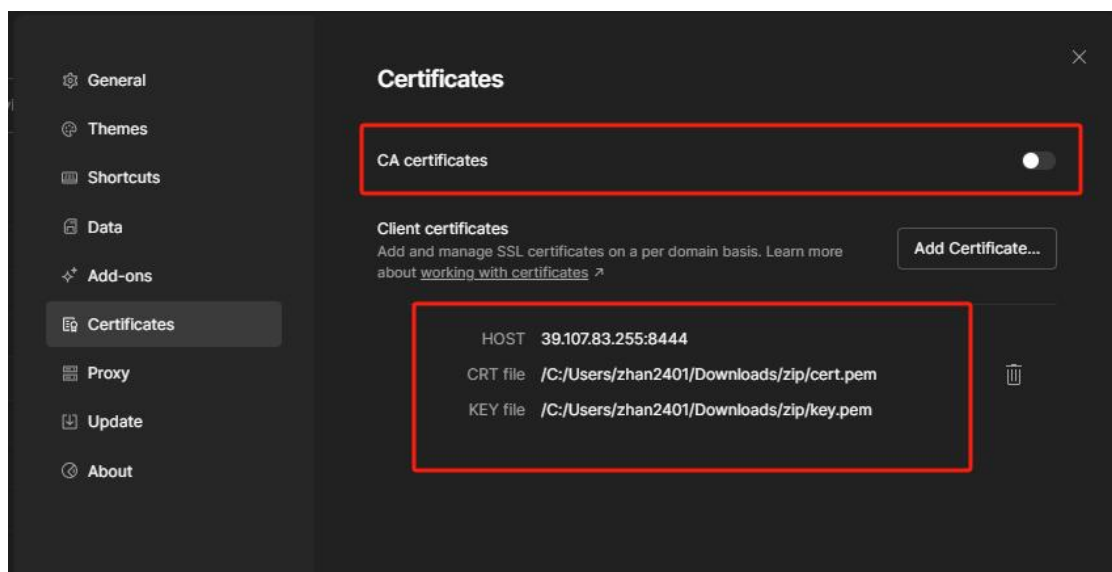
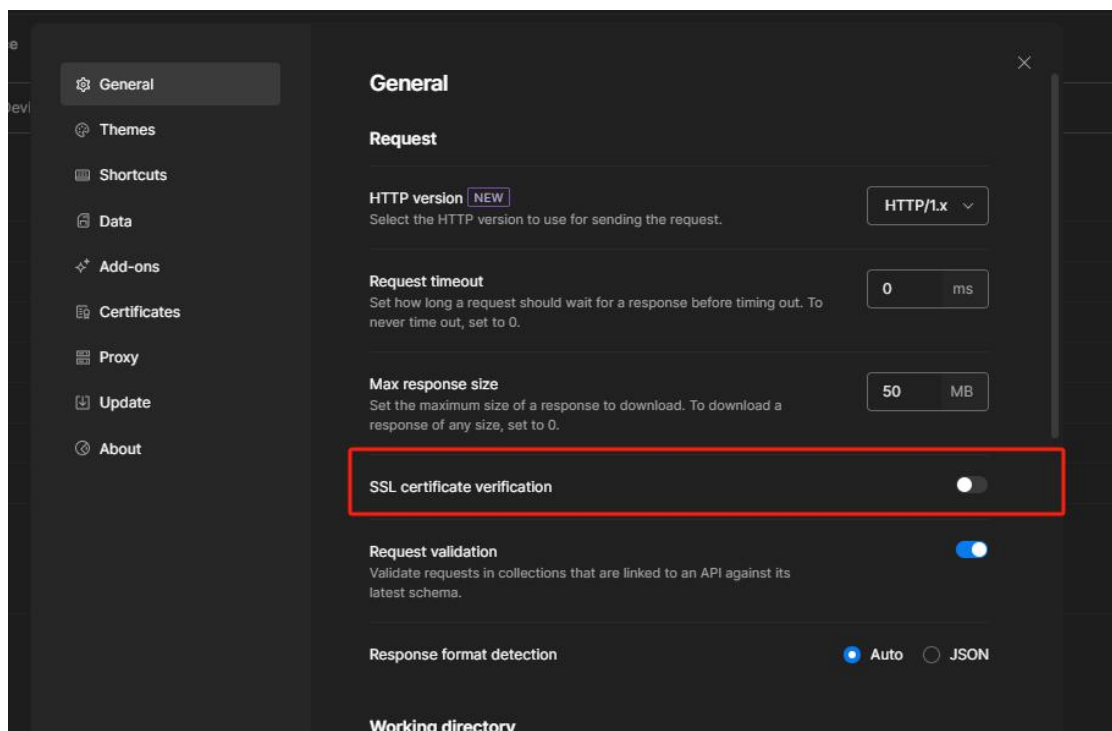
● 参数说明:

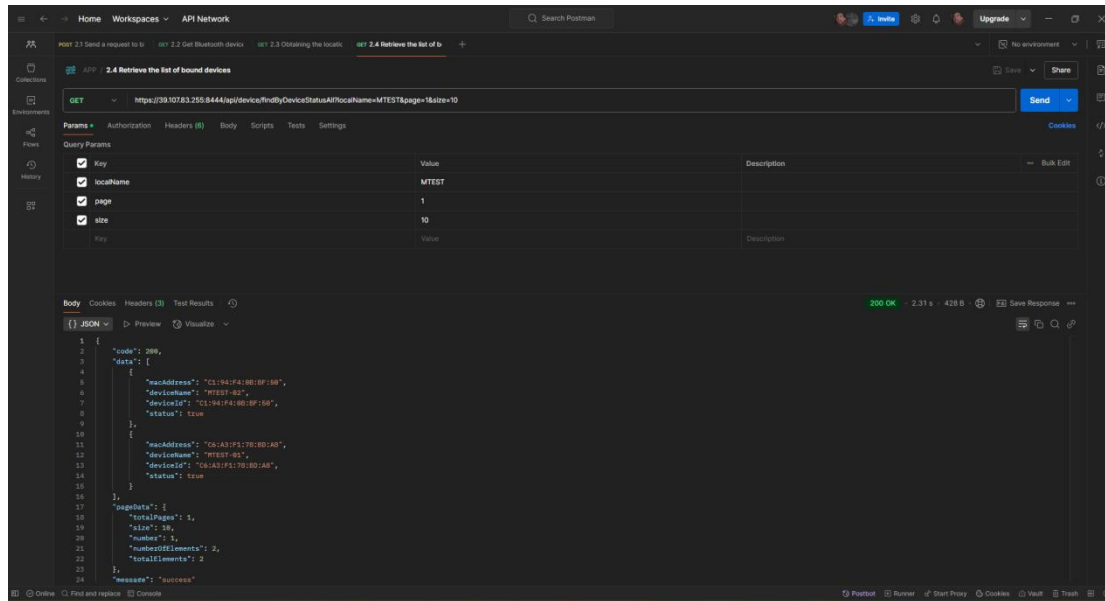
参数名	参数说明	参数类型	是否必需
page	当前页数(默认第一页)	整数	是
size	当前页条数(默认每页 10 条)	整数	是
localName	统一设备烧录 localName	整数	是

响应参数名	响应参数说明	参数类型
macAddress	设备 mac 地址	字符串
deviceName	设备名称	字符串
deviceId	设备 id	字符串
status	设备状态	布尔值
totalPages	总页数	整数

size	请求页_条数	整数
number	请求页数	整数
numberOfElements	当前页返回条数	整数
totalElements	总条数	整数

● PostMan 操作图，如下：





● Example Request

`https://39.107.83.255:8444/api/device/findByDeviceStatusAll?localName=MTEST&page=1&size=10`

● Example Success Response

```

{
  "code": 200,
  "data": [
    {
      "macAddress": "C1:94:F4:0B:BF:50",
      "deviceName": "MTEST-02",
      "deviceId": "C1:94:F4:0B:BF:50",
      "status": true
    },
    {
      "macAddress": "C6:A3:F1:78:BD:A8",
      "deviceName": "MTEST-01",
      "deviceId": "C6:A3:F1:78:BD:A8",
      "status": true
    }
  ],
  "pageData": {
    "totalPages": 1,
    "size": 10,
    "number": 1,

```

```
    "numberOfElements": 2,  
    "totalElements": 2  
  },  
  "message": "success"  
}
```

● Example Failure Response

```
{  
  "code": 500,  
  "data": [],  
  "message": "not bind Device Data"  
}  
  
{  
  "code": 500,  
  "data": [],  
  "message": "Request Param Cannot Be Empty"  
}  
  
{  
  "code": 500,  
  "data": [],  
  "message": "Not Found (bad URL)"  
}  
  
{  
  "code": 401,  
  "data": [],  
  "message": "Unauthorized (this area requires you to log in)"  
}  
  
{  
  "code": 405,  
  "data": [],  
  "message": "Method Not Allowed (wrong HTTP method)"  
}  
  
{  
  "code": 404,  
  "data": [],  
  "message": "Not Found (bad URL)"  
}  
  
{  
  "code": 400,  
  "data": [],  
  "message": "Bad Request (generic user error/bad data/invalid data)"  
}
```

```

}
{
  "code": 500,
  "data": [],
  "message": "Bad Request (generic localName error/bad data/invalid data)"
}
{
  "code": 500,
  "data": [],
  "message": "Request Param Cannot Be Empty"
}
{
  "code": 500,
  "data": [],
  "message": "MAC address is invalid, and the account is not managed"
}

```

2.5 设备请求解除绑定

● 请求格式描述:

HTTP Method	POST
URI	/api/device/receiveDeviceRelieve
Header	Accept: application/json Content-Type: application/json
httpsAgent	提供 cert.pem 证书、key.pem 私钥
URL Params	None
Request (multipart/form-data)	<pre> { "macAddress": "xxxxxx", "localName": "xxxxxx" } </pre>
Response	<pre> { "code": xxx, "data": [], "message": "xxx" } </pre>

● 功能描述:

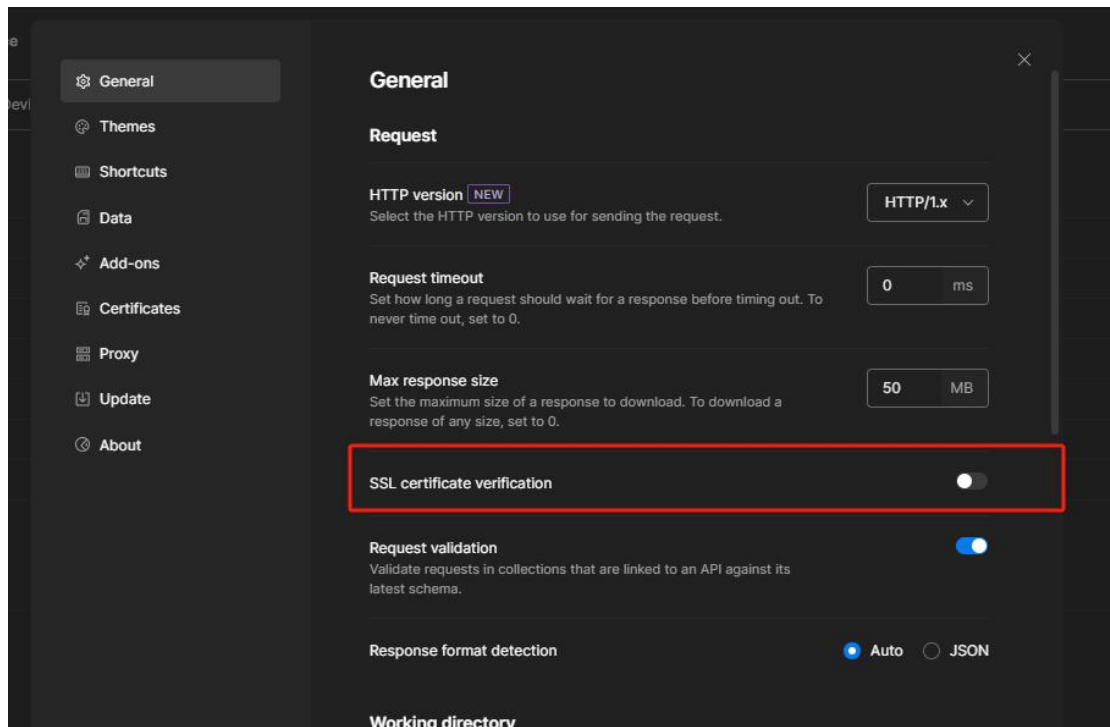
C. 请求头 cert.pem 证书和 key.pem 私钥;

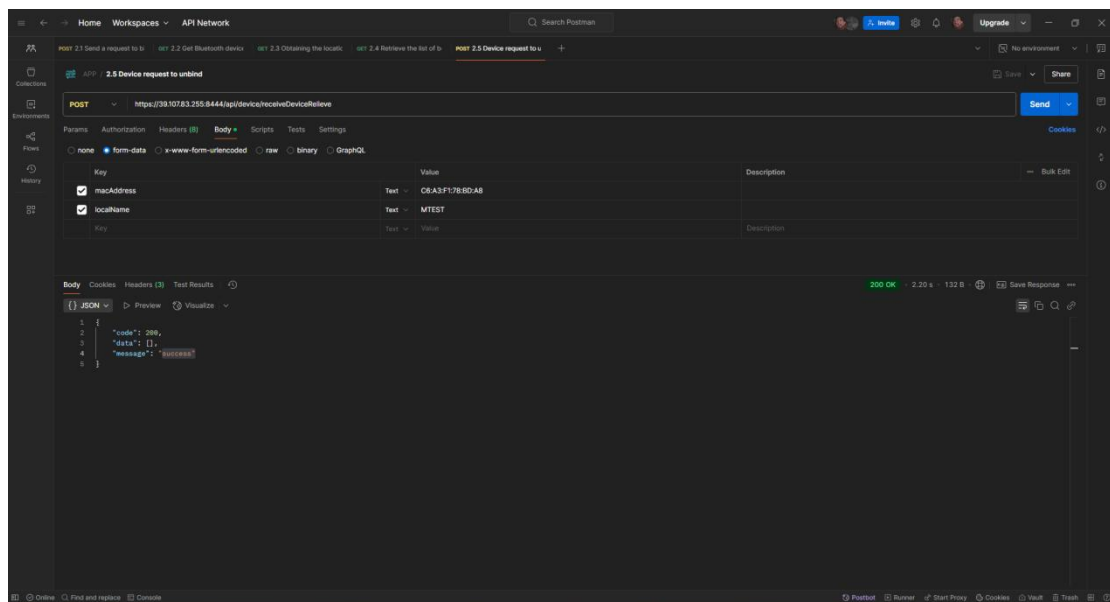
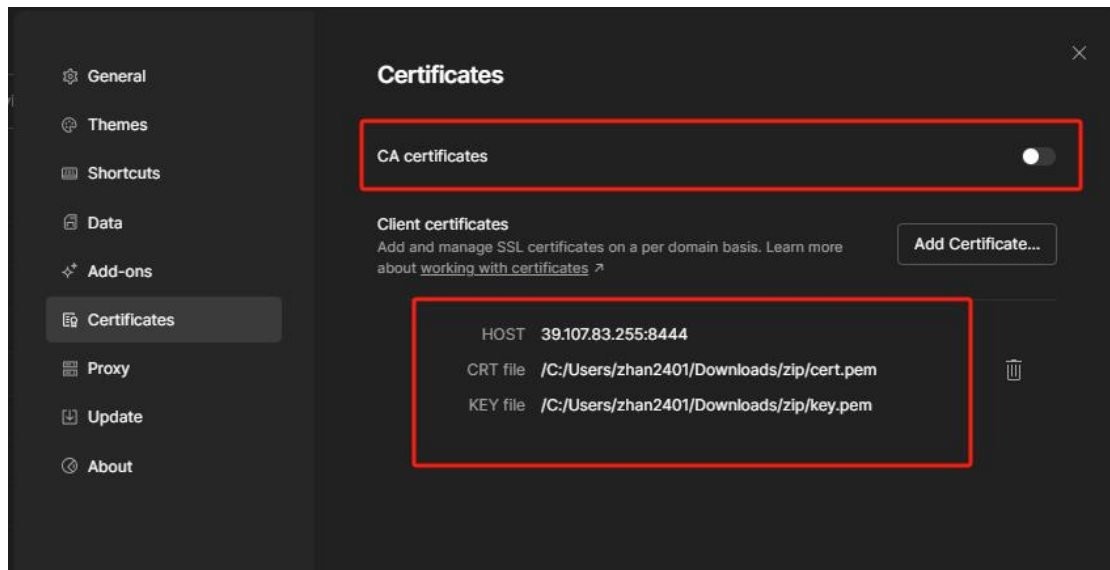
A. 支持在该证书账号下，将单设备解除绑定，恢复无绑定状态;

● 参数说明:

参数名	参数说明	参数类型	是否必需
macAddress	设备 mac 地址	字符串	是
localName	统一设备烧录 localName	字符串	是

● PostMan 操作图，如下:





● Example Request

`https://39.107.83.255:8444/api/device/receiveDeviceRelieve`

● Example Success Response

```
{
  "code": 200,
  "data": [],
  "message": "success"
}
```

● Example Failure Response

```
{
  "code": 500,
  "data": [],
  "message": "Device not found"
}
{
  "code": 500,
  "data": [],
  "message": "The device has been unbound"
}
{
  "code": 500,
  "data": [],
  "message": "Request Param Cannot Be Empty"
}
{
  "code": 500,
  "data": [],
  "message": "Not Found (bad URL)"
}
{
  "code": 401,
  "data": [],
  "message": "Unauthorized (this area requires you to log in)"
}
{
  "code": 405,
  "data": [],
  "message": "Method Not Allowed (wrong HTTP method)"
}
{
  "code": 404,
  "data": [],
  "message": "Not Found (bad URL)"
}
{
  "code": 400,
  "data": [],
  "message": "Bad Request (generic user error/bad data/invalid data)"
}
{
```

```

    "code": 500,
    "data": [],
    "message": "Bad Request (generic localName error/bad data/invalid data)"
  }
  {
    "code": 500,
    "data": [],
    "message": "Request Param Cannot Be Empty"
  }
  {
    "code": 500,
    "data": [],
    "message": "MAC address is invalid, and the account is not managed"
  }
}

```

4. 代码对接事例

3.1 Node. JS 语言代码对接

注：GET 请求

```

const https = require('https');
const fs = require('fs');
// 证书路径
const cert_certificate = 'pem/cert.pem';
const key_certificate = 'pem/key.pem';
// 读取证书和密钥
const cert = fs.readFileSync(cert_certificate);
const key = fs.readFileSync(key_certificate);
// 设置 HTTPS 请求的选项
const options = {
  hostname: '39.107.83.255',
  port: 8444,
  path:
'/api/device/findByBoundDeviceLocationData?localName=MTEST&time=72&page=1&size=10',
  method: 'GET',
  cert: cert,
  key: key,
  rejectUnauthorized: false,
  headers: {
    'User-Agent': 'Node.js',

```

```

        'Accept': 'application/json',
        'Content-Type': 'application/json'
    }
};
// 发送请求
const req = https.request(options, (res) => {
    let data = '';
    res.on('data', (chunk) => {data += chunk;});
    res.on('end', () => { console.log('Response:', data);});
});
// 错误处理
req.on('error', (e) => {console.error('Error:', e);});
// 结束请求
req.end();

```

注：POST 请求

```

const https = require('https');
const fs = require('fs');
const FormData = require('form-data'); // 引入 form-data 模块
// 证书路径
const cert_certificate = 'pem/cert.pem';
const key_certificate = 'pem/key.pem';
// 读取证书和密钥
const cert = fs.readFileSync(cert_certificate);
const key = fs.readFileSync(key_certificate);
// 创建 FormData 实例
const form = new FormData();
form.append('deviceName', 'MTEST-01');
form.append('deviceId', 'C6:A3:F1:78:BD:A8');
form.append('macAddress', 'C6:A3:F1:78:BD:A8');
form.append('localName', MTEST);
// 设置 HTTPS 请求的选项
const options = {
    hostname: '39.107.83.255',
    port: 8444,
    path: '/api/device/receiveDeviceDataBind', // 设置路径
    method: 'POST', // 使用 POST 方法
    cert: cert, // 使用证书
    key: key, // 使用密钥
    rejectUnauthorized: false,
    headers: {
        'User-Agent': 'Node.js', // 请求头用户代理
        'Accept': 'application/json', // 接受的响应类型
        "Content-Type": "multipart/form-data", // 设置为 multipart/form-data
    }
}

```

```

        "Content-Length": form.getLengthSync(), // 添加 Content-Length
        ...form.getHeaders() // 这里 form.getHeaders() 会自动处理 Content-Type 和
        Content-Length
    }
};
// 发送请求
const req = https.request(options, (res) => {
    let data = "";
    res.on('data', (chunk) => {data += chunk;});
    res.on('end', () => {console.log('Response:', data);});
});
// 错误处理
req.on('error', (e) => {console.error('Error:', e);});
// 将表单数据管道到请求体
form.pipe(req);

```

3.2 JAVA 语言代码对接

注：GET 请求

```

import org.bouncycastle.openssl.PEMParser;
import org.bouncycastle.asn1.pkcs.PrivateKeyInfo;
import org.bouncycastle.openssl.jcajce.JcaPEMKeyConverter;
import javax.net.ssl.*;
import java.io.*;
import java.net.*;
import java.security.*;
import java.security.cert.*;

public class HttpRequestExample {
    public static void main(String[] args) throws Exception {
        // 设置证书和私钥路径
        String certFilePath = "pem/cert.pem";
        String keyFilePath = "pem/key.pem";

        URL url = new
URL("https://39.107.83.255:8444/api/device/findByBoundDeviceLocationData?localName=MTEST&time=1&page=
1&size=10");

        // 1. 加载证书
        CertificateFactory cf = CertificateFactory.getInstance("X.509");
        URL certUrl = HttpRequestExample.class.getClassLoader().getResource(certFilePath);
        try (InputStream certInputStream = certUrl.openStream()) {
            X509Certificate cert = (X509Certificate) cf.generateCertificate(certInputStream);

```

```

// 2. 加载私钥
URL keyUrl = HttpRequestExample.class.getClassLoader().getResource(keyFilePath);
PrivateKey privateKey = null;
try (PEMParser pemParser = new PEMParser(new FileReader(keyUrl.getFile())) {
    Object object = pemParser.readObject();
    if (object instanceof PrivateKeyInfo) {
        PrivateKeyInfo privateKeyInfo = (PrivateKeyInfo) object;
        privateKey = new JcaPEMKeyConverter().getPrivateKey(privateKeyInfo);
    } else {
        throw new IllegalArgumentException("Invalid key format");
    }
}

// 3. 创建 KeyStore 并加载证书和私钥
KeyStore keyStore = KeyStore.getInstance("PKCS12");
keyStore.load(null, null); // 创建空的 KeyStore
keyStore.setKeyEntry("alias", privateKey, null, new X509Certificate[]{cert});

// 4. 初始化 KeyManagerFactory
KeyManagerFactory keyManagerFactory = KeyManagerFactory.getInstance("SunX509");
keyManagerFactory.init(keyStore, null);

// 5. 创建一个信任所有证书的 TrustManager
TrustManager[] trustAllCertificates = new TrustManager[]{
    new X509TrustManager() {
        public X509Certificate[] getAcceptedIssuers() { return null; }
        public void checkClientTrusted(X509Certificate[] certs, String authType) {}
        public void checkServerTrusted(X509Certificate[] certs, String authType) {}
    }
};

// 6. 关闭主机名校验
HostnameVerifier allHostsValid = (hostname, session) -> true;

// 7. 初始化 SSLContext
SSLContext sslContext = SSLContext.getInstance("TLS");
sslContext.init(keyManagerFactory.getKeyManagers(), trustAllCertificates, new SecureRandom());

// 8. 创建连接并设置 SSLContext
HttpsURLConnection connection = (HttpsURLConnection) url.openConnection();
connection.setSSLSocketFactory(sslContext.getSocketFactory());
connection.setHostnameVerifier(allHostsValid);

// 9. 设置请求方法和请求头
connection.setRequestMethod("GET");
connection.setRequestProperty("Accept", "application/json");
connection.setRequestProperty("Content-Type", "application/json");
connection.setDoOutput(false); // 不发送请求体

// 10. 获取响应代码
int responseCode = connection.getResponseCode();
System.out.println("Response Code: " + responseCode);

// 11. 打印响应内容 (可选)

```

```

        try (BufferedReader in = new BufferedReader(new
InputStreamReader(connection.getInputStream()))) {
            String inputLine;
            StringBuilder response = new StringBuilder();
            while ((inputLine = in.readLine()) != null) {
                response.append(inputLine);
            }
            System.out.println("Response: " + response.toString());
        }
        // 12. 关闭连接
        connection.disconnect();
        System.out.println("Connection closed");
    } catch (IOException e) {
        e.printStackTrace();
    }
}
}
}
}

```

注：POST 请求

```

import com.google.gson.JsonObject;
import org.bouncycastle.asn1.pkcs.PrivateKeyInfo;
import org.bouncycastle.openssl.PEMParser;
import org.bouncycastle.openssl.jcajce.JcaPEMKeyConverter;

import javax.net.ssl.*;
import java.io.*;
import java.net.URL;
import java.nio.charset.StandardCharsets;
import java.security.KeyStore;
import java.security.PrivateKey;
import java.security.SecureRandom;
import java.security.cert.CertificateFactory;
import java.security.cert.X509Certificate;

public class ReceiveDeviceRelieve {
    public static void main(String[] args) throws Exception {
        // 用户信息，证书、密钥和参数
        String certFilePath = "pem/cert.pem";
        String keyFilePath = "pem/key.pem";
        String localName = "MTEST-01";
        String macAddress = "C6:A3:F1:78:BD:A8";
        // 构造 URL
        URL url = new URL("https://39.107.83.255:8444/api/device/receiveDeviceRelieve");
    }
}

```

```

// 加载证书
CertificateFactory cf = CertificateFactory.getInstance("X.509");
InputStream certInputStream =
ReceiveDeviceRelieve.class.getClassLoader().getResourceAsStream(certFilePath);
if (certInputStream == null) {
    throw new IOException("Certificate file not found: " + certFilePath);
}
X509Certificate cert = (X509Certificate) cf.generateCertificate(certInputStream);

// 加载私钥
InputStream keyInputStream =
ReceiveDeviceRelieve.class.getClassLoader().getResourceAsStream(keyFilePath);
if (keyInputStream == null) {
    throw new IOException("Key file not found: " + keyFilePath);
}
PEMParser pemParser = new PEMParser(new InputStreamReader(keyInputStream));
PrivateKey privateKey = null;
Object object = pemParser.readObject();
if (object instanceof PrivateKeyInfo) {
    PrivateKeyInfo privateKeyInfo = (PrivateKeyInfo) object;
    privateKey = new JcaPEMKeyConverter().getPrivateKey(privateKeyInfo);
} else {
    throw new IllegalArgumentException("Invalid key format");
}

// 创建 KeyStore 并加载证书和私钥
KeyStore keyStore = KeyStore.getInstance("PKCS12");
keyStore.load(null, null); // 创建空的 KeyStore
keyStore.setKeyEntry("alias", privateKey, null, new X509Certificate[]{cert});

// 初始化 KeyManagerFactory
KeyManagerFactory keyManagerFactory = KeyManagerFactory.getInstance("SunX509");
keyManagerFactory.init(keyStore, null);

// 创建一个信任所有证书的 TrustManager
TrustManager[] trustAllCertificates = new TrustManager[]{
    new X509TrustManager() {
        public X509Certificate[] getAcceptedIssuers() { return null; }
        public void checkClientTrusted(X509Certificate[] certs, String authType) {}
        public void checkServerTrusted(X509Certificate[] certs, String authType) {}
    }
};

```

```

HostnameVerifier allHostsValid = (hostname, session) -> true;

// 初始化 SSLContext, 使用 TrustManager
SSLContext sslContext = SSLContext.getInstance("TLS");
sslContext.init(keyManagerFactory.getKeyManagers(), trustAllCertificates, new SecureRandom());

// 创建连接并设置 SSLContext
HttpsURLConnection connection = (HttpsURLConnection) url.openConnection();
connection.setSSLSocketFactory(sslContext.getSocketFactory());
connection.setHostnameVerifier(allHostsValid);

// 设置请求方法和请求头
connection.setRequestMethod("POST");
connection.setRequestProperty("Accept", "application/json");
connection.setRequestProperty("Content-Type", "application/json"); // 设置为 JSON 请求体
connection.setDoOutput(true);

// 构造 JSON 请求体
JsonObject requestBody = new JsonObject();
requestBody.addProperty("localName", localName);
requestBody.addProperty("macAddress", macAddress);

// 发送请求体
try (OutputStream os = connection.getOutputStream()) {
    String string = requestBody.toString();
    byte[] input = string.getBytes(StandardCharsets.UTF_8);
    os.write(input, 0, input.length);
}

// 获取响应代码
int responseCode = connection.getResponseCode();
System.out.println("Response Code for " + localName + ": " + responseCode);

// 打印响应内容
try (BufferedReader in = new BufferedReader(new InputStreamReader(connection.getInputStream()))) {
    String inputLine;
    StringBuilder response = new StringBuilder();
    while ((inputLine = in.readLine()) != null) {
        response.append(inputLine);
    }
    System.out.println("Response for " + localName + ": " + response.toString());
}

// 关闭连接

```

```

        connection.disconnect();
        System.out.println("Connection closed for " + localName);
    }
}

```

3.3 Python 语言代码对接

注：GET 请求

```

import requests

# 证书路径
cert = '/pem/cert.pem' # 客户端证书
key = '/pem/key.pem' # 客户端私钥
# 请求 URL
url =
'https://39.107.83.255:8444/api/device/findByBoundDeviceLocationData?localName=MTEST&time=72&page=1&size=10' # 服务端接口
# 设置请求头
headers = {
    'User-Agent': 'Python/Requests',
    'Custom-Header': 'value'
}
# 发起请求
try:
    # 使用 requests 库发起请求，传入客户端证书和私钥
    response = requests.get(
        url,
        headers=headers,
        cert=(cert, key), # 传入客户端证书和私钥
        verify=False
    )
    # 输出服务器响应
    if response.status_code == 200:
        print("请求成功，服务器响应： ")
        print(response.text)
    else:
        print(f"请求失败，状态码： {response.status_code}")
except requests.exceptions.SSLError as e:
    print(f"SSL 错误: {e}")
except requests.exceptions.RequestException as e:
    print(f"请求失败: {e}")

```

注：POST 请求

```
import requests
import json

# 证书路径
cert = 'E:/JAVA_DATE/Java_Project/MMSemiconductor/MMSemiconductor/src/main/resources/pem/cert11.pem'
# 客户端证书
key = 'E:/JAVA_DATE/Java_Project/MMSemiconductor/MMSemiconductor/src/main/resources/pem/key11.pem'
# 客户端私钥

# 请求 URL
url = 'https://39.107.83.255:8444/api/device/receiveDeviceRelieve' # 服务端接口

# 设置请求头
headers = {
    'User-Agent': 'Python/Requests',
    'Content-Type': 'application/json', # 设置为 JSON 格式
    'Custom-Header': 'value'
}

# 构建请求的 JSON 数据
payload = {
    'macAddress': 'C6:A3:F1:78:BD:A8',
    'localName': 'MTEST-01'
}

# 发起 POST 请求，进行 SSL/TLS 双向认证
try:
    # 使用 requests 库发起请求，传入客户端证书和私钥
    response = requests.post(
        url,
        headers=headers,
        data=json.dumps(payload), # 使用 json.dumps() 将字典转换为 JSON 字符串
        cert=(cert, key), # 传入客户端证书和私钥
        verify=False
    )

    # 输出服务器响应
    if response.status_code == 200:
        print("请求成功，服务器响应：")
        print(response.text)
    else:
        print(f"请求失败，状态码：{response.status_code}")
```

```
except requests.exceptions.SSLError as e:
    print(f"SSL 错误: {e}")
except requests.exceptions.RequestException as e:
    print(f"请求失败: {e}")
```

3.4 C# 语言代码对接

注：GET 请求

```
using System;
using System.Net.Http;
using System.Security.Cryptography.X509Certificates;
using System.Net.Security;
using System.Threading.Tasks;

class Program
{
    static async Task Main(string[] args)
    {
        // 创建 HttpClientHandler 来设置证书和 TLS 协议
        var handler = new HttpClientHandler
        {
            // 设置 TLS 协议版本为 TLS 1.2
            SslProtocols = System.Security.Authentication.SslProtocols.Tls12
        };
        // PFX 证书路径
        string pfxFilePath = @"C:\Users\001\Downloads\certificate.pfx";
        try
        {
            // 加载 PFX 证书
            X509Certificate2 cert = LoadPfxCertificate(pfxFilePath);
            // 将证书添加到 HTTP 客户端处理器
            handler.ClientCertificates.Add(cert);
            handler.ServerCertificateCustomValidationCallback = (sender, cert, chain,
sslPolicyErrors) => {
                if (sslPolicyErrors == System.Net.Security.SslPolicyErrors.None) {
                    return true;
                }
                return true;
            };
            // 创建 HttpClient, 并将 HttpClientHandler 作为参数传入
            using (var client = new HttpClient(handler))
            {
```

```

        // 设置请求地址和端口
        var requestUri =
"https://39.107.83.255:8444/api/device/findByBoundDeviceLocationData?localName=MTEST&page=1&size=10";

        try
        {
            // 发送 GET 请求
            HttpResponseMessage response = await client.GetAsync(requestUri);
            // 确保成功响应
            response.EnsureSuccessStatusCode();
            // 获取响应内容并打印
            string responseContent = await response.Content.ReadAsStringAsync();
            Console.WriteLine("Response: " + responseContent);
        }
        catch (Exception ex)
        {
            Console.WriteLine("Request failed: " + ex.Message);
        }
    }
}

catch (Exception ex)
{
    Console.WriteLine("Error loading certificate: " + ex.Message);
}
}

// 使用 Import 加载证书的方法
private static X509Certificate2 LoadPfxCertificate(string pfxFilePath)
{
    try
    {
        // 使用 X509Certificate2 加载 PFX 文件
        #pragma warning disable SYSLIB0057
        return new X509Certificate2(pfxFilePath);
        #pragma warning restore SYSLIB0057
    }
    catch (Exception ex)
    {
        throw new Exception("Error loading PFX certificate: " + ex.Message);
    }
}
}
}

```

注：POST 请求

```
using System;
using System.Net.Http;
using System.Security.Cryptography.X509Certificates;
using System.Net.Security;
using System.Threading.Tasks;

class Program
{
    static async Task Main(string[] args)
    {
        // 创建 HttpClientHandler 来设置证书和 TLS 协议
        var handler = new HttpClientHandler
        {
            // 设置 TLS 协议版本为 TLS 1.2
            SslProtocols = System.Security.Authentication.SslProtocols.Tls12
        };
        // PFX 证书路径
        string pfxFilePath = @"C:\Users\zhan2401\Downloads\certificate.pfx";
        try
        {
            // 加载 PFX 证书
            X509Certificate2 cert = LoadPfxCertificate(pfxFilePath);
            // 将证书添加到 HTTP 客户端处理器
            handler.ClientCertificates.Add(cert);
            handler.ServerCertificateCustomValidationCallback = (sender, cert, chain,
sslPolicyErrors) => {
                if (sslPolicyErrors == System.Net.Security.SslPolicyErrors.None) {
                    return true;
                }
                return true;
            };
            // 创建 HttpClient, 并将 HttpClientHandler 作为参数传入
            using (var client = new HttpClient(handler))
            {
                // 设置请求地址和端口
                var requestUri =
                "https://192.168.3.113:8444/api/device/receiveDeviceRelieve";

                try
                {
                    // 创建表单数据
                    var form = new FormUrlEncodedContent(new[]
                    {
```

```

        new KeyValuePair<string, string>("macAddress", "D4:94:03:E9:53:33"),
        new KeyValuePair<string, string>("localName", "MTEST")
    });

    // 设置请求头
    client.DefaultRequestHeaders.Accept.ParseAdd("application/json");

    // 发送 POST 请求
    var response = await client.PostAsync(requestUri, form);

    // 处理响应
    if (response.IsSuccessStatusCode)
    {
        string responseContent = await response.Content.ReadAsStringAsync();
        Console.WriteLine("Response: " + responseContent);
    }
    else
    {
        Console.WriteLine($"Request failed with status code: {response.StatusCode}");
    }
}

catch (Exception ex)
{
    Console.WriteLine("Request failed: " + ex.Message);
}

}

catch (Exception ex)
{
    Console.WriteLine("Error loading certificate: " + ex.Message);
}

}

// 使用 Import 加载证书的方法
private static X509Certificate2 LoadPfxCertificate(string pfxFilePath)
{
    try
    {
        // 使用 X509Certificate2 加载 PFX 文件
        #pragma warning disable SYSLIB0057
        return new X509Certificate2(pfxFilePath);
        #pragma warning restore SYSLIB0057
    }
    catch (Exception ex)

```

```

    {
        throw new Exception("Error loading PFX certificate: " + ex.Message);
    }
}
}

```

Status Codes

HTTP Code	Error Mapping	Description
200	success	请求成功
200	Already Bound	已绑定
500	Device not found	未找到设备
500	Device is already bound	设备已绑定
500	Not Found (bad URL)	未找到（错误 URL）
400	Bad Request (generic user error/bad data/invalid data)	
401	Unauthorized (this area requires you to log in)	
405	Method Not Allowed (wrong HTTP method)	

500	Request Param Cannot Be Empty	
500	not Device Data	
500	parameter error	
500	not bind Device Data	
500	The device has been unbound	
500	MAC address is invalid, and the account is not managed	